

```
"""
```

Уранічний бот – щоденні підказки на час ретроградного Урана.

Модель монетизації (MVP, без інтеграції платіжного шлюзу):

1. Користувач тисне /subscribe і бачить реквізити для оплати (Monobank jar / Li
2. Після оплати користувач тисне «Я оплатив» – заявка йде адміну.
3. Адмін підтверджує командою /approve <user_id>, після чого бот щодня надсилає

Це навмисно просто: не потребує акаунту продавця в Stripe (складно з України), можна запустити за один вечір. Пізніше легко замінити крок 2-3 на Telegram Stars або LiqPay/Fondy API для автоматизації.

```
"""
```

```
import json
import logging
import os
import random
import sqlite3
from datetime import datetime, time
from pathlib import Path

from aiogram import Bot, Dispatcher, F
from aiogram.filters import Command
from aiogram.types import Message, CallbackQuery, InlineKeyboardMarkup, InlineKey
from aiogram.utils.keyboard import InlineKeyboardBuilder
from apscheduler.schedulers.asyncio import AsyncIOScheduler
import asyncio

# ----- НАЛАШТУВАННЯ -----
# Ці значення беруться зі змінних середовища (Variables в Railway),
# щоб не редагувати код напряму. Локально можна задати через файл .env
# або просто підставити текст замість os.getenv(...) значень.
BOT_TOKEN = os.getenv("BOT_TOKEN", "ВСТАВ_СЮДИ_TOKEN_ВІД_BOTFATHER")
ADMIN_ID = int(os.getenv("ADMIN_ID", "123456789")) # твій Telegram user_id (дізн
PAYMENT_INFO = os.getenv(
    "PAYMENT_INFO",
    "👉 Оплата підписки – 99 грн / місяць\n\n"
    "Monobank jar: https://send.monobank.ua/jar/ВСТАВ_ПОСИЛАННЯ\n"
    "або LiqPay: https://liqpay.ua/ВСТАВ_ПОСИЛАННЯ\n\n"
    "Після оплати натисни кнопку «Я оплатив » нижче.",
)

DAILY_SEND_TIME = time(hour=9, minute=0) # о котрій надсилати підказку щодня
DB_PATH = Path(__file__).parent / "subscribers.db"
TIPS_PATH = Path(__file__).parent / "tips.json"
```

```

logging.basicConfig(level=logging.INFO)
bot = Bot(token=BOT_TOKEN)
dp = Dispatcher()

# ----- БАЗА ДАНИХ -----
def init_db():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS subscribers (
            user_id INTEGER PRIMARY KEY,
            username TEXT,
            status TEXT DEFAULT 'pending',    -- pending | active | inactive
            joined_at TEXT,
            tip_index INTEGER DEFAULT 0
        )
    """)
    conn.commit()
    conn.close()

def add_pending(user_id: int, username: str):
    conn = sqlite3.connect(DB_PATH)
    conn.execute(
        "INSERT OR IGNORE INTO subscribers (user_id, username, status, joined_at)
        (user_id, username, datetime.now().isoformat()),
    )
    conn.commit()
    conn.close()

def set_status(user_id: int, status: str):
    conn = sqlite3.connect(DB_PATH)
    conn.execute("UPDATE subscribers SET status=? WHERE user_id=?", (status, user_id))
    conn.commit()
    conn.close()

def get_active_subscribers():
    conn = sqlite3.connect(DB_PATH)
    rows = conn.execute("SELECT user_id, tip_index FROM subscribers WHERE status='active'")
    conn.close()
    return rows

def bump_tip_index(user_id: int, new_index: int):

```

```

conn = sqlite3.connect(DB_PATH)
conn.execute("UPDATE subscribers SET tip_index=? WHERE user_id=?", (new_index,))
conn.commit()
conn.close()

def load_tips():
    with open(TIPS_PATH, encoding="utf-8") as f:
        return json.load(f)

TIPS = load_tips()

# ----- ХЕНДЛЕРИ -----
@dp.message(Command("start"))
async def cmd_start(message: Message):
    text = (
        "🍌 Привіт! Я – Уранічний бот.\n\n"
        "Ретроградний Уран стартує 6-10 вересня 2026 і триватиме до лютого 2027."
        "Я надсилатиму тобі коротку щоденну підказку, як пережити цей період "
        "усвідомлено, без хаосу.\n\n"
        "Команди:\n"
        "/subscribe – оформити підписку\n"
        "/tip – отримати підказку зараз (демо, одна безкоштовна)\n"
        "/status – перевірити статус підписки"
    )
    await message.answer(text)

@dp.message(Command("tip"))
async def cmd_tip_demo(message: Message):
    tip = random.choice(TIPS)
    await message.answer(f"🧠 {tip}")

@dp.message(Command("subscribe"))
async def cmd_subscribe(message: Message):
    add_pending(message.from_user.id, message.from_user.username or "")
    kb = InlineKeyboardBuilder()
    kb.add(InlineKeyboardButton(text="Я оплатив ✅", callback_data="paid"))
    await message.answer(PAYMENT_INFO, reply_markup=kb.as_markup())

@dp.callback_query(F.data == "paid")
async def cb_paid(callback: CallbackQuery):
    user = callback.from_user

```

```

await callback.message.answer("Дякую! Заявку передано адміну, підписка активу
await bot.send_message(
    ADMIN_ID,
    f"
```

```

for user_id, _ in get_active_subscribers():
    try:
        await bot.send_message(user_id, text)
    except Exception as e:
        logging.warning(f"Не вдалось надіслати {user_id}: {e}")
await message.answer("Розіслано.")

# ----- ЩОДЕННА РОЗСИЛКА -----
async def send_daily_tips():
    for user_id, tip_index in get_active_subscribers():
        idx = tip_index % len(TIPS)
        tip = TIPS[idx]
        try:
            await bot.send_message(user_id, f"🌅 Ранкова уранічна підказка:\n\n💡
            bump_tip_index(user_id, idx + 1)
        except Exception as e:
            logging.warning(f"Не вдалось надіслати {user_id}: {e}")

async def main():
    init_db()
    scheduler = AsyncIOScheduler(timezone="Europe/Kyiv")
    scheduler.add_job(send_daily_tips, "cron", hour=DAILY_SEND_TIME.hour, minute=
    scheduler.start()
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())

```